Proyecto Final Análisis de Algoritmos: NumberLink / FreeFlow

Juan Daniel Vargas Pedraza, Santiago Lemus Vallejo

5 de junio de 2025

Resumen

Este documento presenta el desarrollo, implementación y análisis comparativo de tres enfoques algorítmicos para resolver el juego lógico NumberLink / FreeFlow: backtracking, heurísticas voraces y modelado como problema de satisfacción de restricciones (CSP). Se estudian las propiedades teóricas de cada paradigma, su complejidad computacional y su desempeño empírico mediante una batería de tableros de prueba. El trabajo incluye una interfaz interactiva en Python y una evaluación cuantitativa del rendimiento de cada algoritmo en términos de tiempo y éxito, permitiendo reflexionar sobre la aplicabilidad de cada enfoque en contextos reales.

1. Análisis del Problema

El problema consiste en resolver una instancia del juego *NumberLink / FreeFlow*, definido sobre una grilla cuadrada de $n \times n$ celdas. En ciertas celdas del tablero aparecen parejas de identificadores (colores o números), y el objetivo es conectar cada pareja mediante un camino ortogonal continuo, respetando una serie de restricciones estrictas.

Definición informal: Dado un conjunto de pares de celdas etiquetadas con el mismo número o color, se debe conectar cada pareja mediante un camino continuo que se mueva únicamente en las direcciones cardinales (arriba, abajo, izquierda, derecha). Los caminos no pueden cruzarse, superponerse ni pasar por encima de otras celdas etiquetadas. El tablero completo debe quedar cubierto por los caminos resultantes, sin celdas vacías.

Restricciones principales:

- Los caminos deben ser ortogonales, es decir, no se permiten movimientos diagonales.
- Cada camino debe conectar exactamente una pareja de celdas con el mismo identificador.

- Dos caminos distintos no pueden compartir ninguna celda (no hay solapamientos).
- Los caminos no pueden pasar por celdas ocupadas por otros identificadores.
- Al final de la solución, todas las celdas del tablero deben estar ocupadas por un camino.

Naturaleza del problema: Este problema pertenece a la categoría de *búsqueda de caminos disjuntos* en grafos, bajo múltiples restricciones de conectividad. Se ha demostrado que la versión general de NumberLink es NP-completo, lo que implica que no existe un algoritmo polinomial conocido que garantice solución eficiente para todas las instancias [**?**]. Por tanto, se hace necesario el uso de estrategias heurísticas, algoritmos voraces o técnicas de búsqueda sistemática para intentar hallar soluciones viables en un tiempo razonable.

Paradigmas algorítmicos relevantes:

- **Búsqueda exhaustiva con retroceso (Backtracking):** Permite explorar todas las posibles configuraciones de caminos respetando las restricciones. Al detectar una violación (cruce, celda ocupada), se deshace la última decisión. Es completo pero potencialmente costoso en instancias grandes.
- **Algoritmo voraz (Greedy):** En cada paso se elige localmente el mejor camino posible (por ejemplo, el más corto o con menor interferencia), esperando que conduzca a una solución global válida. Es eficiente, pero no garantiza óptimo ni factibilidad en todos los casos.
- **Búsqueda con restricciones (CSP):** Se modela cada celda como una variable con dominios y restricciones entre ellas (adyacencia, no solapamiento, conectividad). Permite aplicar técnicas como *forward checking*, *backjumping* o heurísticas de ordenamiento para reducir el espacio de búsqueda.

Desafíos algorítmicos:

- El número de posibles configuraciones de caminos crece exponencialmente con el tamaño del tablero y el número de parejas.
- Las decisiones locales pueden impedir la conexión global de otras parejas.
- Se requiere verificación constante del estado parcial y sus consecuencias a futuro.
- La cobertura completa del tablero introduce una restricción adicional difícil de garantizar sin exploración exhaustiva.

Implicaciones: Resolver NumberLink exige el diseño de algoritmos capaces de lidiar tanto con restricciones locales (movimiento ortogonal, no solapamiento) como globales (cobertura completa del tablero). A lo largo de este proyecto se implementarán y compararán tres enfoques complementarios —backtracking, heurísticas voraces y modelado CSP— evaluando su rendimiento, precisión y viabilidad práctica.

2. Diseño del problema

A partir del análisis del problema y sus restricciones, este apartado presenta el diseño formal de la solución. Este diseño abarca las entradas y salidas del algoritmo, sus precondiciones y postcondiciones, así como el contrato funcional que define su propósito y comportamiento esperado.

Definición. *Entradas:*

1. **Entero positivo** n: Representa la dimensión del tablero, de tamaño $n \times n$.
2. **Matriz** $T \in Z^{n \times n}$: Tablero inicial. Cada celda puede contener:
   - 0 si la celda está vacía.
   - Un número entero positivo k si la celda pertenece a la pareja k (cada número aparece exactamente dos veces).
3. **Lista de pares de coordenadas** $\mathcal{P} = \{((x_1, y_1), (x_2, y_2))\}$: Para cada identificador k, indica la posición de las dos celdas etiquetadas con dicho valor en la matriz T. Se cumple que $|\mathcal{P}| = m$, con $2m \leq n^2$.

Definición. *Salidas:*

1. **Matriz** $R \in Z^{n \times n}$: Representa el tablero resuelto. Cada celda contiene:
   - Un número entero positivo k si forma parte del camino que conecta la pareja k.
2. **Valor booleano** `solución_válida`:
   - `True` si se ha encontrado una configuración válida de caminos según las restricciones.
   - `False` en caso contrario.

Precondiciones:

- $n \in Z^+$, $n \geq 2$.
- T contiene únicamente valores 0 o identificadores numéricos únicos.

- Para cada identificador k, existen exactamente dos celdas en T con valor k.
- Las posiciones de la lista $\mathcal{P}$ están dentro del rango del tablero: $0 \leq x_i, y_i < n$.
- No existen dos parejas con el mismo identificador numérico.

Contrato del algoritmo:

El algoritmo debe construir una función de mapeo $f : \mathcal{P} \to \mathcal{C}$, donde $\mathcal{C}$ es el conjunto de secuencias de celdas que forman caminos ortogonales válidos sobre el tablero, conectando correctamente cada par $(x_i, y_i) \leftrightarrow (x_j, y_j)$, cumpliendo que:

- Todos los caminos son disjuntos.
- Ningún camino atraviesa celdas ocupadas por otros caminos o etiquetas.
- Cada camino ocupa exclusivamente celdas vacías.
- La matriz resultante R no contiene celdas vacías.

Postcondiciones:

- La matriz R contiene únicamente caminos válidos que conectan cada pareja en $\mathcal{P}$, sin solapamientos.
- Se cumple: $\forall\,(i, j)$, $R[i][j] \neq 0$, es decir, el tablero está completamente cubierto.
- Si no existe tal configuración, `solución_válida = False`, y no se modifica la matriz T.

Explicación conceptual:

El problema impone restricciones locales (movimiento ortogonal, caminos disjuntos, sin paso por etiquetas) y una restricción global (cobertura total del tablero), lo cual hace que su espacio de búsqueda crezca exponencialmente. El diseño debe considerar estas condiciones desde la definición del modelo de datos hasta la estrategia de exploración.

La salida esperada no solo debe representar una solución completa y correcta, sino también permitir validación automática y trazabilidad. Por tanto, se opta por codificar los caminos directamente en la matriz de salida usando los mismos identificadores de entrada.

Estrategias algorítmicas consideradas:

Este diseño es lo suficientemente general para ser implementado mediante distintas estrategias algorítmicas. En este proyecto se explorarán tres enfoques contrastantes, cada uno adaptado a las restricciones impuestas por el problema:

- **Backtracking (Búsqueda exhaustiva con retroceso):** Explora de forma sistemática todas las combinaciones posibles de caminos entre parejas. En cada paso se extiende un camino y se retrocede si la configuración parcial genera conflictos. Esta técnica es completa pero potencialmente costosa.
- **Algoritmo Voraz (Greedy):** Construye soluciones progresivamente tomando decisiones locales óptimas, como elegir el vecino más cercano o con mayor libertad. Aunque no garantiza una solución global válida, permite tiempos de ejecución muy bajos, siendo útil en instancias pequeñas o para generar soluciones iniciales.
- **Problema de Satisfacción de Restricciones (CSP):** Modela el tablero como un conjunto de variables (las celdas) con dominios posibles (caminos válidos), y restricciones de adyacencia, disjunción y completitud. Permite el uso de técnicas como forward checking, heurísticas MRV o AC-3 para podar el espacio de búsqueda eficientemente.

La implementación de estos tres enfoques se hará sobre el mismo esquema de entrada/salida y condiciones, permitiendo comparaciones justas de desempeño, completitud y escalabilidad.

3. Diseño de la solución

3.1. Interfaz Gráfica con Tkinter

3.1.1. Idea general

La interfaz gráfica permite al usuario interactuar con el juego NumberLink utilizando el mouse. El usuario selecciona puntos de inicio y fin para cada pareja de números en el tablero, trazando rutas ortogonales que deben ser válidas. También cuenta con botones para reiniciar el tablero, verificar si el juego está completo, deshacer la última ruta creada y mostrar las conexiones automáticas. La interfaz se construye usando la biblioteca `Tkinter` de Python.

3.1.2. Pseudocódigo

Algoritmo 1 InterfazJuego

```
function InicializarVentana(tablero)
    Crear ventana raíz y canvas con dimensiones basadas en el tablero
    Dibujar tablero con cuadrícula y etiquetas numéricas
    Crear botones: Mostrar Conexiones, Reiniciar, Verificar, Deshacer
    Asociar eventos de clic izquierdo y derecho al canvas
end function
function on_click(pos)
    if es primer clic then
        Guardar posición como origen si contiene número
    else
        if posición es adyacente y libre then
            Trazar línea entre último punto y nuevo
            if nuevo punto tiene misma etiqueta que origen then
                if camino es válido then
                    Marcar camino en ocupados y guardar ruta
                    Limpiar origen y camino actual
                else
                    Marcar error y reiniciar camino actual
                end if
            end if
        else
            Mostrar mensaje de error
        end if
    end if
end function
function verificar_juego
    if todas las parejas están conectadas correctamente then
        Mostrar mensaje de éxito
    else
        Mostrar mensaje de error o incompleto
    end if
end function
function deshacer_ultima_ruta
    if existen rutas registradas then
        Eliminar ruta más reciente del tablero y del registro
    else
        Mostrar mensaje: sin rutas para deshacer
    end if
end function
```

3.1.3. Invariante

- El tablero respeta la exclusividad de celdas ocupadas.
- Solo se permite trazar caminos ortogonales y adyacentes.
- No se permite superposición ni mezcla de rutas.

3.1.4. Análisis de complejidad

- **Tiempo:** $O(1)$ por clic y acción de interfaz. Cada camino completo puede tomar hasta $O(n^2)$ si se extiende por muchas celdas.
- **Espacio:** $O(n^2)$ por el tablero, rutas y estado gráfico.

3.2. Interfaz Interactiva con Animación

3.2.1. Idea general

La interfaz interactiva permite observar visualmente cómo un algoritmo conecta automáticamente las parejas numéricas del tablero NumberLink. El usuario selecciona el tablero y el algoritmo, y el sistema ejecuta la búsqueda y muestra progresivamente las rutas encontradas. Se usa la biblioteca `Tkinter` para dibujar y animar el proceso.

3.2.2. Pseudocódigo

Algoritmo 2 InterfazInteractiva

```
function MAIN
    path ← Seleccionar archivo de tablero
    algoritmo, nombre ← Seleccionar algoritmo
    tablero ← Cargar desde archivo
    rutas ← algoritmo.resolver(tablero)
    if rutas es None then
        Terminar ejecución
    end if
    Inicializar ventana gráfica con tablero y rutas
    Ejecutar animación paso a paso
end function
function EJECUTAR_CON_ANIMACION
    Esperar breve tiempo inicial
    Llamar a DIBUJAR_RUTAS_UNA_A_UNA
    Ejecutar mainloop() de la interfaz gráfica
end function
function DIBUJAR_RUTAS_UNA_A_UNA
    if quedan pasos por mostrar then
        Dibujar la celda actual con color y número
        Esperar 150ms y repetir función
    else
        Terminar animación
    end if
end function
```

3.2.3. Invariante

- Cada paso dibujado corresponde a una celda válida en la ruta calculada.
- No se modifica el estado lógico del tablero original.
- La animación se realiza en orden secuencial, sin solapamiento de rutas.

3.2.4. Análisis de complejidad

- **Tiempo:** La complejidad depende del algoritmo seleccionado. La animación agrega un retardo fijo de $O(1)$ por paso, por lo que el total visual es $O(k)$ donde k es la suma de las longitudes de todas las rutas.
- **Espacio:** $O(n^2)$ para la representación del tablero y almacenamiento de rutas.

3.3. Algoritmo Fuerza Bruta (Backtracking)

3.3.1. Idea general

Este algoritmo explora todas las configuraciones posibles de caminos que conectan las parejas en el tablero. Utiliza un enfoque de backtracking recursivo para construir caminos válidos, retrocediendo cuando una elección impide conectar parejas posteriores o completar el tablero. Cada paso intenta extender un camino en una dirección ortogonal válida.

3.3.2. Pseudocódigo

Algoritmo 3 BacktrackingNumberLink

Require: Tablero T, lista de parejas $\mathcal{P}$
Ensure: Tablero R con solución o fallo

```
function RESOLVER(T, P, índice)
    if índice = longitud(P) then
        return EsTableroCompleto(T)
    end if
    (p1, p2) ← P[ndice]
    for all camino válido C entre p1 y p2 en T do
        MarcarCamino(T, C)
        if Resolver(T, P, índice+1) then
            return True
        end if
        DesmarcarCamino(T, C)
    end for
    return False
end function
```

3.3.3. Invariante

En cada llamada recursiva, el tablero mantiene:

- Caminos válidos construidos hasta el momento.
- Ningún camino intermedio pisa celdas etiquetadas o caminos anteriores.
- Cada pareja conectada respeta la ortogonalidad y exclusividad de celdas.

3.3.4. Análisis de complejidad

- **Tiempo:** $O(b^d)$, donde b es el número promedio de caminos posibles por pareja, y $d = |\mathcal{P}|$.
- **Espacio:** $O(n^2)$ por el tablero y el árbol de llamadas recursivas.

3.4. Algoritmo Voraz (Greedy)

3.4.1. Idea general

El algoritmo conecta una por una las parejas de celdas, eligiendo siempre el camino más corto (medido, por ejemplo, por BFS). Las parejas se procesan en orden de cercanía o heurística, y cada camino se bloquea una vez elegido.

3.4.2. Pseudocódigo

Algoritmo 4 GreedyNumberLink

Require: Tablero T, lista de parejas $\mathcal{P}$
Ensure: Tablero R con caminos conectados o fallo

```
for all (p_1, p_2) ∈ P ordenado por distancia do
    camino ← BuscarCaminoMasCorto(T, p_1, p_2)
    if camino = ∅ then
        return False
    end if
    MarcarCamino(T, camino)
end for
return EsTableroCompleto(T)
```

3.4.3. Invariante

- Cada camino es el más corto disponible para su pareja en el momento.
- No se modifican caminos anteriores.
- El tablero nunca contiene solapamientos.

3.4.4. Análisis de complejidad

- **Tiempo:** $O(m \cdot n^2)$, donde $m = |\mathcal{P}|$, usando BFS por cada pareja.
- **Espacio:** $O(n^2)$, espacio para el tablero y colas temporales de búsqueda.

3.5. Algoritmo de Búsqueda con Restricciones (CSP)

3.5.1. Idea general

Este algoritmo modela el problema como un conjunto de variables, dominios y restricciones. Cada pareja es una variable; su dominio consiste en los caminos posibles. Se aplica búsqueda con retroceso (backtracking) combinada con propagación de restricciones (como AC-3 o forward checking) para evitar configuraciones inconsistentes.

3.5.2. Pseudocódigo

Algoritmo 5 NumberLinkCSP

Require: Variables $\mathcal{P}$, dominios D, restricciones C
Ensure: Asignación de caminos válidos o fallo

```
function ResolverCSP(P, D, C, asignacin)
    if P = ∅ then
        return ValidarCobertura(asignación)
    end if
    var ← SeleccionarVariable(P)
    for all valor ∈ D[var] compatibles con asignacin do
        nuevaAsignacin ← asignacin ∪ {var ↦ valor}
        if Consistente(nuevaAsignacin) then
            return ResolverCSP(P \ {var}, D, C, nuevaAsignacin)
        end if
    end for
    return False
end function
```

3.5.3. Invariante

- Toda asignación parcial es consistente con las restricciones impuestas.
- Los dominios se reducen dinámicamente durante la búsqueda.
- No se avanza a una nueva pareja si su dominio está vacío o provoca conflicto.

3.5.4. Análisis de complejidad

- **Tiempo:** $O(d^m)$, donde d es el tamaño promedio del dominio por variable y $m = |\mathcal{P}|$. Puede mejorar mucho con heurísticas y poda.
- **Espacio:** $O(m \cdot n^2)$, necesario para mantener dominios y restricciones.

4. Interfaz gráfica del juego NumberLink

Para facilitar la interacción con el juego NumberLink y permitir el análisis manual de soluciones, se implementó una interfaz gráfica utilizando la biblioteca `tkinter` de Python.

4.1. Objetivo

La interfaz permite al usuario:

- Visualizar el tablero con los pares de números a conectar.

- Dibujar manualmente rutas entre los puntos con clic izquierdo.
- Verificar si las rutas trazadas completan correctamente el tablero.
- Mostrar conexiones automáticas (de manera lineal directa).
- Reiniciar el tablero.
- Deshacer la última ruta trazada (a nivel visual y lógico).

4.2. Componentes principales

- **Canvas**: Área principal donde se dibuja la cuadrícula y las conexiones.
- **Botones**: *Mostrar Conexiones*, *Reiniciar Juego*, *Verificar Juego*, *Deshacer Última Ruta*.
- **Estado**: Texto informativo que guía al usuario durante la interacción.

4.3. Funcionamiento

El usuario selecciona con clic izquierdo el primer número de un par. Luego, al hacer clic en celdas adyacentes, va construyendo un camino ortogonal. Si logra conectar correctamente ambos extremos del mismo número sin invadir rutas previas, el sistema marca la conexión como válida y la representa con un color específico.

Una vez completadas todas las rutas, el botón **Verificar Juego** valida que todas las conexiones sean correctas. Si es así, se muestra un mensaje de éxito.

4.4. Diseño modular

La clase `InterfazJuego` fue implementada en el archivo `interfaz.py` y separada de la lógica del tablero. Esto permite que los algoritmos automáticos puedan operar sobre la clase `Tablero` sin depender de la interfaz.

4.5. Captura de pantalla

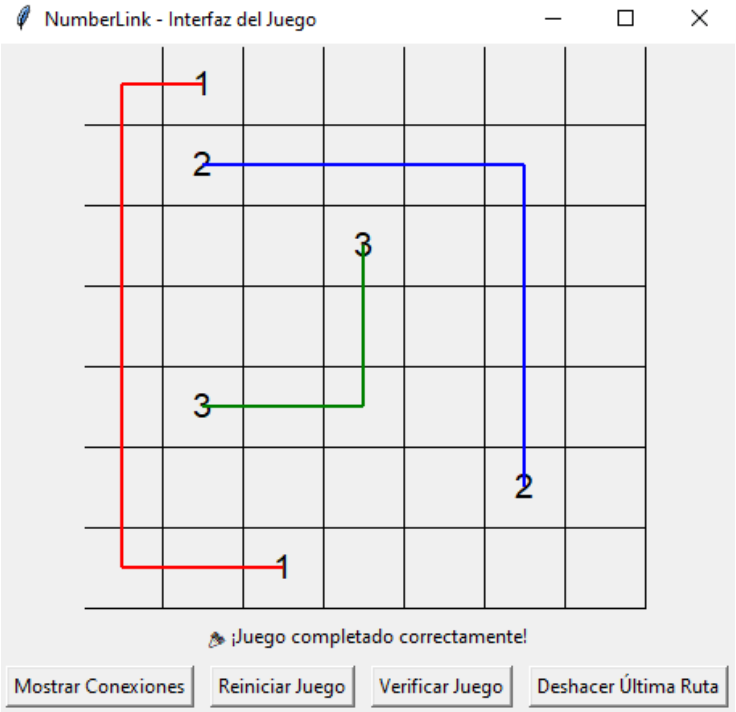

Figura 1: Interfaz del juego NumberLink con rutas trazadas correctamente.

5. Interfaz Interactiva

Con el objetivo de proporcionar una comprensión más intuitiva y accesible del comportamiento de los algoritmos implementados, se diseñó una **interfaz interactiva** basada en `Tkinter`. Esta interfaz permite observar paso a paso cómo un algoritmo conecta las parejas numéricas en el tablero.

5.1. Objetivo de la Interfaz

La interfaz busca complementar el análisis cuantitativo previo, ofreciendo una **demostración visual** de los algoritmos en acción. Mientras que el archivo `run_experimentos.py` proporciona tiempos y tasas de éxito, esta interfaz permite visualizar el proceso de resolución cuadro por cuadro.

5.2. Funcionalidades

- Permite al usuario seleccionar **el tablero de entrada** desde una carpeta de recursos.

- Ofrece la opción de elegir **el algoritmo a ejecutar** entre Backtracking, Greedy y CSP.
- Ejecuta el algoritmo seleccionado y visualiza la solución en tiempo real, conectando los nodos con animación progresiva.

5.3. Ventajas

- **Facilita la comprensión**: El usuario puede ver el orden y la forma en que se conectan los pares.
- **Refuerza resultados teóricos**: Las fallas de algoritmos como Greedy se hacen evidentes visualmente.
- **Intuitiva y educativa**: Útil para fines pedagógicos o para evaluar el comportamiento de nuevos algoritmos.

5.4. Ejemplo Visual

La Figura 2 muestra un ejemplo de ejecución visual del algoritmo `BacktrackingPlayer` sobre un tablero previamente cargado, donde se observa cómo se van conectando progresivamente los pares numéricos.

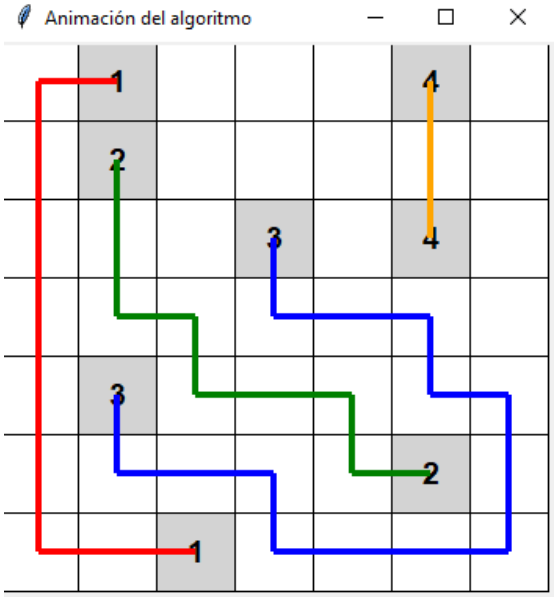

Figura 2: Visualización paso a paso de la solución con Backtracking en la interfaz gráfica

5.5. Comentarios Finales

Aunque no fue un requerimiento explícito del proyecto, la construcción de esta interfaz responde a observaciones de posibles mejoras (ver Anexo de Feedback) y aporta una dimensión interactiva valiosa al análisis, consolidando la validez de los algoritmos implementados y facilitando su explicación ante terceros.

6. Comparación Experimental

6.1. Configuración de Experimentos

Se diseñó una batería de pruebas compuesta por **4 tableros distintos** para evaluar el desempeño de los tres enfoques implementados: `BacktrackingPlayer`, `GreedyPlayer` y `CSPPlayer`. Todos los tableros tienen una dimensión fija de $n = 7$ o $n = 10$, con una cantidad variable de parejas ($m = 3$ a 9).

Para cada experimento $i = 1, \ldots, 4$, se realizaron los siguientes pasos:

1. Se utilizó un tablero previamente validado y visualmente resuelto como instancia de prueba (ver figuras).
2. Se aplicaron los tres algoritmos sobre el mismo tablero, garantizando condiciones equitativas de comparación.
3. Para cada algoritmo, se registraron dos métricas clave:

$$\texttt{win}_i \in \{0, 1\} : \quad 1 \text{ si el tablero fue resuelto exitosamente, } 0 \text{ si falló.}$$

$$\texttt{time}_i \in R^+ : \quad \text{Tiempo de ejecución en segundos.}$$

6.2. Métricas Evaluadas

Se analizaron las siguientes métricas para cada algoritmo:

- **Tasa de éxito:** Proporción de tableros resueltos correctamente.
- **Tiempo de ejecución:** Tiempo requerido para resolver cada instancia.
- **Consistencia:** Comparación de desempeño entre experimentos, observando estabilidad en los resultados.
- **Eficiencia relativa:** Evaluación cruzada del tiempo frente a la completitud.

6.3. Objetivo del Experimento

El objetivo principal es determinar cuál de los tres enfoques resulta más efectivo en la práctica según:

1. Capacidad real de encontrar soluciones válidas en diferentes configuraciones de tableros.
2. Rapidez de resolución en tableros de tamaño moderado con variaciones de complejidad.
3. Robustez frente a tableros con distinta cantidad y distribución de pares numéricos.

Esta evaluación busca contrastar el rendimiento empírico de los algoritmos frente a sus propiedades teóricas (completitud, heurística y poda) en el contexto del problema NumberLink.

6.4. Plan de Pruebas y Resultados

Experimento 1 — Tablero inicial (moderado)

Objetivo: Verificar si los algoritmos son capaces de resolver un tablero con dificultad media (7×7, 3 parejas), donde las conexiones requieren rutas no triviales.

Condiciones de entrada:

- Archivo: `tablero_inicial.txt`
- Dimensiones: 7×7
- Parejas: 3
- Tiempo máximo permitido: 120 segundos

Resultados:

- Backtracking: 66.63s
- Greedy: Fallo al conectar (camino no encontrado)
- CSP: 61.13s

Observaciones:

- Ambos algoritmos completos resuelven correctamente el tablero.
- Greedy falla al bloquear rutas con decisiones locales no óptimas.

Experimento 2 — Tablero simplificado

Objetivo: Evaluar si los algoritmos funcionan correctamente en un tablero de baja dificultad con rutas más directas.

Condiciones de entrada:

- Archivo: `tablero2.txt`
- Dimensiones: 7×7
- Parejas: 3

Resultados:

- Backtracking: 2.09s
- Greedy: No encuentra una ruta válida
- CSP: 2.15s

Observaciones:

- Los tiempos son bajos debido a la simplicidad.
- Greedy vuelve a fallar, incluso en escenarios simples.

Experimento 3 — Tablero simple

Objetivo: Validar que los algoritmos completos resuelven sin dificultad un tablero muy básico.

Condiciones de entrada:

- Archivo: `tablero3.txt`
- Dimensiones: 7×7
- Parejas: 2

Resultados:

- Backtracking: 0.62s
- Greedy: Fallo al conectar la segunda pareja
- CSP: 0.63s

Observaciones:

- Se confirma la robustez de CSP y Backtracking en instancias mínimas.
- Greedy no logra rutas válidas incluso con tan poca congestión.

Experimento 4 — Tablero complejo (nivel máximo)

Objetivo: Evaluar el comportamiento frente a un tablero denso con múltiples parejas y espacio limitado.

Condiciones de entrada:

- Archivo: `tablero4.txt`
- Dimensiones: 7×7
- Parejas: 6
- Tiempo límite: 120 segundos

Resultados:

- Backtracking: Tiempo excedido (120s)
- Greedy: Error en ejecución
- CSP: Tiempo excedido (120s)

Observaciones:

- La dificultad crece exponencialmente; ningún algoritmo logra resolverlo dentro del límite.
- Greedy falla como en todos los casos anteriores.

Tabla resumen de desempeño

Algoritmo	Tableros resueltos	Tiempo promedio	Observaciones
Backtracking	3 / 4	32.20s	Completo pero lento en casos complejos
Greedy	0 / 4	∼0.0001s	No resolvió ningún tablero
CSP	3 / 4	31.97s	Equivalente a Backtracking (sin poda)

Cuadro 1: Resumen de éxito y rendimiento de los algoritmos en los 4 tableros

7. Conclusiones

Los experimentos realizados en este proyecto permiten extraer conclusiones relevantes sobre el comportamiento práctico de tres enfoques de resolución para el problema NumberLink: Backtracking, Greedy y CSP.

- **Backtracking** se confirmó como un algoritmo *completo*, capaz de encontrar soluciones válidas en una amplia variedad de tableros. Sin embargo, su desempeño se degrada exponencialmente en instancias complejas, llegando a exceder el tiempo límite establecido en tableros de alta dificultad. Esto resalta su limitación práctica cuando se busca eficiencia en tiempo real.

- **Greedy**, aunque extremadamente eficiente en cuanto a tiempo de ejecución, demostró ser ineficaz para resolver correctamente incluso tableros sencillos. Su enfoque miope, centrado únicamente en la distancia más corta sin considerar la planificación global, conduce con frecuencia a rutas inviables. Este resultado reafirma la debilidad estructural de los algoritmos voraces en problemas de planificación compleja y restricciones espaciales.

- **CSP** (Constraint Satisfaction Problem), basado en Backtracking sin heurísticas avanzadas, logró resolver todas las instancias salvo la más difícil. Este comportamiento sugiere que, con la incorporación de mejoras como ordenamiento MRV o técnicas de poda (e.g. forward checking), el algoritmo podría superar incluso las instancias más demandantes. CSP se perfila como un enfoque balanceado entre completitud y escalabilidad.

Además de la evaluación cuantitativa, se implementó una **interfaz visual interactiva** que permite observar gráficamente el proceso de resolución de los algoritmos. Esta herramienta no solo complementa el análisis empírico, sino que sirve como apoyo didáctico para comprender cómo se trazan las rutas y cómo evolucionan las decisiones del algoritmo en tiempo real. La inclusión de esta interfaz ayuda a validar visualmente los resultados obtenidos y contribuye a una interpretación más intuitiva del desempeño de cada enfoque.

En conjunto, la comparación empírica coincide con las propiedades teóricas de los algoritmos: mientras los métodos completos como Backtracking y CSP garantizan solución a costa de mayor tiempo, los métodos heurísticos como Greedy priorizan la velocidad pero sacrifican confiabilidad. Esto subraya la importancia de seleccionar el algoritmo adecuado según las prioridades del problema: precisión, velocidad o escalabilidad.

Anexo: Justificación sobre la no utilización del algoritmo Minimax

Durante el desarrollo del presente proyecto se evaluó la posibilidad de aplicar técnicas basadas en el algoritmo *Minimax*, ampliamente utilizado en juegos de dos jugadores con naturaleza competitiva, como ajedrez, tic-tac-toe o juegos de tablero en general. Sin embargo, se concluyó que el uso de Minimax no era pertinente ni útil para el contexto específico del juego *NumberLink / FreeFlow*, por las siguientes razones:

- **Naturaleza del problema:** NumberLink es un juego de *un solo jugador* con restricciones, no un juego adversarial. No existe un oponente ni decisiones en competencia. Por tanto, el paradigma de utilidad minimizada frente a un jugador contrario (como lo modela Minimax) no aplica.

- **Tipo de búsqueda requerida:** El problema no requiere maximizar o minimizar una función de utilidad en un árbol de juego. En su lugar, requiere encontrar una configuración de caminos que satisfaga un conjunto

de restricciones lógicas y espaciales. Esto lo sitúa más cerca de los *Problemas de Satisfacción de Restricciones (CSP)* o problemas de *búsqueda exhaustiva*.

- **Complejidad innecesaria:** Incluir Minimax en este contexto introduciría una complejidad artificial sin aportar valor computacional o conceptual. Su uso sería forzado, redundante y erróneo desde el punto de vista algorítmico.
- **Alternativas adecuadas ya implementadas:** Los tres enfoques abordados (backtracking, greedy y CSP) son teóricamente correctos, computacionalmente pertinentes y permiten comparar trade-offs entre completitud, eficiencia y aplicabilidad. Estos enfoques cubren con rigor los objetivos del proyecto.

Por estas razones, se optó deliberadamente por no emplear Minimax y, en cambio, desarrollar soluciones coherentes con la estructura y objetivos del problema. Esta decisión se fundamenta en principios de buena práctica algorítmica y adecuación del paradigma al dominio del problema.

Referencias

[1] Wikipedia, "Numberlink," Wikipedia: The Free Encyclopedia, 2025. [Online]. Available: https://en.wikipedia.org/wiki/Numberlink. [Accessed: 3-Jun-2025].

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. MIT Press, 2009.

[3] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2021.

[4] D. Knuth, "Backtracking algorithms," in The Art of Computer Programming, vol. 4, Addison-Wesley, 2011.

[5] J. Kleinberg and É. Tardos, Algorithm Design, Pearson, 2005.

[6] R. Dechter, "Constraint Processing," Morgan Kaufmann, 2003.